Capítulo 1

Errores de redondeo y conceptos básicos del análisis numérico

Introducción

Consideremos el siguiente programa:

```
a=1/3;
b=1-a;
b=b-a;
b=b-a;
if b==0
  disp('verdadero')
else
  disp('falso')
  b
endif
```

El procedimiento es el siguiente: asignamos a una variable el resultado de la operación 1 dividido 3. Luego restamos de 1 ese valor y repetimos esa resta otras 2 veces más. Si el resultado es cero, imprimimos verdadero, caso contrario imprimimos falso y mostramos el valor *distinto de cero* que se obtendría.

Si ejecutamos el script vemos que ocurre un *error*, ya que, desde el punto de vista matemático, la operación debería dar 0. Esto ocurre debido a que las operaciones las ejecuta una computadora.

En este capítulo veremos por qué ocurren este tipo de errores, los clasificaremos y daremos algunas definiciones y nociones relacionadas a la representación de números en máquinas de aritmética finita. También se tratarán las ideas básicas de los problemas matemáticos y los conceptos de consistencia, estabilidad y convergencia.

1.1. Errores numéricos

El error numérico hace referencia a la diferencia entre un valor calculado y el valor *real*. Esto no significa que el valor calculado sea un valor por equivocación, sino que tiene una diferencia de un valor de referencia o valor real, al cual queremos estimar y no conocemos. Es por ello que se debe estudiar y saber manejar este tipo de errores, para saber qué tan bueno es nuestro valor calculado.

En general, en los problemas de la ingeniería, al realizar aproximaciones se introducen errores que provienen de diferentes fuentes que los generan. Es decir, cuando se quiere realizar un cálculo determinado, y llegar a un valor real, se realizan en el proceso varias aproximaciones, y en ellas introducimos diferentes tipos de errores:

- **Errores por el modelo matemático** propuesto para la formulación del problema (ecuaciones diferenciales y condiciones, suposiciones ideales, etc.).

- **Errores de aproximación de la geometría**, que se realizan al aproximar algún objeto o entorno con figuras o cuerpos geométricos con la finalidad de reducir y facilitar los cálculos.

- **Errores en los datos físicos**. Por ejemplo, las constantes de un material o variables físicas del entorno, o mediciones experimentales de las mismas.

- **Errores de los métodos numéricos de aproximación**. Estos métodos, que son los que estudiaremos en este curso, se utilizan para aproximar el modelo matemático propuesto. Por lo tanto, introduce un error de aproximación a ese modelo que a su vez aproxima al problema de ingeniería a resolver.

- **Errores en la resolución del sistema de ecuaciones**. Por lo general, los métodos numéricos se basan en transformar un modelo matemático en un sistema de ecuaciones. Resolver dicho sistema genera en sus cálculos una importante fuente de errores.

- **Errores de redondeo**. Son los que se abordan en este capítulo. Son los que genera la computadora para realizar los cálculos debido a su aritmética finita.

1.1.1. Error absoluto y relativo

Antes de avanzar con el estudio y análisis de los conceptos relacionados a los errores antes descritos, es importante tener en claro las siguientes definiciones:

Definición 1.1. Sea $\hat{p}$ una aproximación del número no nulo p. El *error absoluto* es

$$E_p = |p - \hat{p}|$$

y el *error relativo* es

$$R_p = \frac{|p - \hat{p}|}{|p|}.$$

En otras palabras, el error absoluto mide la *distancia* entre el valor exacto p y su aproximación $\hat{p}$, mientras que el error relativo mide qué tan grande es ese error en relación al valor exacto.

Con los métodos numéricos en general, se obtiene una aproximación $\hat{p}$ de una solución exacta p. También proporcionan una estimación para el error absoluto E_p. Con estos datos podemos determinar un intervalo en el cual se encuentra el valor exacto p.

Para comprender esto, consideremos el caso en que sabemos que el error absoluto E_p es menor o igual a e_p (el valor e_p es dato, y es lo que denominamos una estimación del error E_p). Entonces

$$\begin{aligned} |p-\hat{p}| =& E_p \leq e_p \\ -e_p \leq& p-\hat{p} \leq e_p \\ \hat{p}-e_p \leq& p \leq \hat{p}+e_p \end{aligned}$$

Luego, conociendo una estimación del error ($E_p \leq e_p$) y una aproximación $\hat{p}$ de la solución, podemos asegurar que el valor exacto p se encuentra a una distancia de $\hat{p}$ a lo sumo de e_p.

Si conocemos $\hat{p}$ y una estimación $E_p \leq e_p$ (es decir, conocemos e_p), se pude demostrar la siguiente estimación para el error relativo R_p.

$$R_p = \frac{|p-\hat{p}|}{|p|} \leq \frac{e_p}{|\hat{p}|}.$$

Siempre que podamos decir que $e_p \ll \hat{p}$.

Muchos algoritmos como los que veremos más adelante se detienen cuando se cumple que $\dfrac{e_p}{|\hat{p}|}$ es menor a una tolerancia preestablecida para el error relativo.

En general, un método numérico para calcular soluciones de ecuaciones provee sucesivas aproximaciones $\hat{p}$ y estimaciones del error absoluto e_p.

Si se desea una aproximación con un error absoluto menor a una tolerancia `tol`, detendremos el procedimiento cuando se cumpla $e_p <$ `tol`.

Si se desea una aproximación con un error relativo menor a una tolerancia `tolr`, detendremos el procedimiento cuando se cumpla $\dfrac{e_p}{|\hat{p}|} <$ `tolr`.

1.2. Errores de redondeo

La representación de un número real en una computadora está limitada a la cantidad finita de cifras que tenga la mantisa, por lo cual, existe una diferencia entre el valor del número real en cuestión y su representación en la computadora. Esta diferencia se denomina **error de redondeo**.

1.2.1. Aritmética de computadoras

Un **sistema de punto flotante** $\mathbb{F}(\beta, t, e_{\text{mín}}, e_{\text{máx}})$ queda determinado por cuatro parámetros:

- β: la **base** del sistema (en las computadoras modernas, $\beta = 2$, en notación decimal $\beta = 10$).

- t: el número de **dígitos de la mantisa** (precisión).

- $e_{\text{mín}}$, $e_{\text{máx}}$: los límites del **exponente**.

Todo número $x \in \mathbb{F}$ distinto de cero se representa en la forma normalizada

$$x = \pm\,(0.d_1 d_2 \ldots d_t)_\beta \times \beta^e, \tag{1.1}$$

donde $d_1 \neq 0$ es el primer dígito de la mantisa y $e_{\text{mín}} \leq e \leq e_{\text{máx}}$. La condición $d_1 \neq 0$ garantiza la unicidad de la representación y se denomina **normalización**.

Ejemplo 1 (Representación exacta: 1/2)**.** El número $x = 0.5$ en base decimal se representa en forma normalizada como

$$x = 0.5 \times 10^0 \quad (\beta = 10),$$

y en base binaria como

$$x = 0.1_2 \times 2^0 \quad (\beta = 2).$$

En ambos casos la representación es **exacta** y finita. Este número pertenece a $\mathbb{F}$ sin ninguna aproximación.

Ejemplo 2 (Representación inexacta: 1/3)**.** El número $x = 1/3$ no tiene representación finita en base decimal:

$$x = 0.\overline{3} \times 10^0 \quad (\beta = 10).$$

Tampoco la tiene en base binaria:

$$x = 0.\overline{01}_2 \times 2^0 \quad (\beta = 2).$$

En ambas bases la representación es **periódica** e infinita, por lo que debe truncarse o redondearse al almacenarse en $\mathbb{F}$. La mantisa dispone de un número finito de elementos (más adelante le diremos *bits* para el caso binario), por lo que la representación se trunca en ese punto introduciendo un error de redondeo. Este error, aunque pequeño ($|\varepsilon| \leq \varepsilon_{\text{maq}}/2$), explica el comportamiento observado en Octave al comienzo del capítulo: la expresión `1 - a - a - a` devuelve `false`, porque `1/3` no se almacena de forma exacta y el error de redondeo se acumula en las tres restas.

Observación 1 (Conjunto finito y discreto)**.** El conjunto $\mathbb{F}$ tiene una cantidad finita de elementos: no todos los números reales son representables. Entre dos números consecutivos de $\mathbb{F}$ existe una brecha, y cualquier número real que caiga en esa brecha debe aproximarse al representable más cercano.

Estándar IEEE 754: doble precisión

El estándar IEEE 754 es el que siguen prácticamente todos los procesadores y lenguajes de programación modernos, incluido Octave. En **doble precisión** (64 bits), los bits se distribuyen de la siguiente manera:

s	exponente	mantisa (fracción)
1 bit	11 bits	52 bits

64 bits en total

- **Signo** (s, 1 bit): $s = 0$ para positivo, $s = 1$ para negativo.
- **Exponente** (11 bits): se almacena con un sesgo (*bias*) de 1023, de modo que el exponente real es $e = e_{\text{almacenado}} - 1023$, con rango $-1022 \leq e \leq 1023$.
- **Mantisa** (52 bits): almacena la parte fraccionaria. Gracias a la normalización, el primer bit de la mantisa es siempre 1 y no necesita almacenarse (*bit implícito*), lo que equivale a tener 53 bits efectivos de precisión.

El valor representado es:

$$x = (-1)^s \times 1.d_1 d_2 \ldots d_{52} \times 2^e.$$

Épsilon de máquina

Definición 1.2 (Épsilon de máquina)**.** El **épsilon de máquina** ε_{maq} es el menor número positivo representable en $\mathbb{F}$ tal que

$$1 + \varepsilon_{\text{maq}} > 1.$$

Equivalentemente, es la distancia entre 1 y el siguiente número representable.

Para doble precisión, con 52 bits de mantisa y el bit implícito:

$$\varepsilon_{\text{maq}} = 2^{-52} \approx 2{,}22 \times 10^{-16}.$$

Esto significa que la doble precisión garantiza aproximadamente 15–16 dígitos decimales significativos. En Octave, este valor es accesible directamente como `eps`:

```
>> eps
ans = 2.2204e-16
>> 1 + eps > 1
ans = 1
>> 1 + eps/2 > 1
ans = 0
```

El épsilon de máquina es la cota del error de redondeo relativo: para cualquier $x \in \mathbb{R}$, su representación $\mathrm{fl}(x) \in \mathbb{F}$ satisface

$$\frac{|x - \mathrm{fl}(x)|}{|x|} \leq \frac{\varepsilon_{\text{maq}}}{2}.$$

Esto conecta directamente con la definición de *dígitos exactos* que daremos más adelante (ver Definición 1.3): trabajar en doble precisión implica disponer de a lo sumo 15–16 dígitos exactos en cualquier cálculo.

Overflow y underflow

El rango del exponente en doble precisión impone límites al tamaño de los números representables:

- **Overflow**: ocurre cuando el resultado de una operación supera el máximo representable $x_{\text{máx}} \approx 1{,}80 \times 10^{308}$. Octave devuelve `Inf` en ese caso.
- **Underflow**: ocurre cuando el resultado es menor en valor absoluto que el mínimo normalizado representable $x_{\text{mín}} \approx 2{,}23 \times 10^{-308}$. Octave devuelve `0`, lo que puede introducir errores silenciosos difíciles de detectar.

Observación 2 (Underflow silencioso)**.** El overflow es fácil de detectar porque `Inf` propaga resultados claramente incorrectos. El underflow es más peligroso: un número muy pequeño se redondea a cero sin advertencia, y los cálculos posteriores continúan sin indicación de error, pudiendo producir resultados incorrectos de forma imperceptible.

1.2.2. Cifras significativas

Definición 1.3 (Dígitos exactos)**.** Se dice que la aproximación $\hat{p}$ de p tiene t **dígitos exactos** (o t *cifras significativas correctas*) si t es el mayor entero no negativo tal que

$$\frac{|p - \hat{p}|}{|p|} \leq 5 \times 10^{-t}.$$

Esta definición cuantifica cuántos dígitos decimales de $\hat{p}$, contados desde el primero dígito no nulo, coinciden con los de p. En particular, t dígitos exactos implica que el error relativo es inferior a media unidad en la t-ésima posición significativa.

Ejemplos

Ejemplo 3 ($\pi \approx 3{,}14$)**.** Tomemos $p = \pi = 3{,}14159\ldots$ y $\hat{p} = 3{,}14$. El error relativo es

$$R_p = \frac{|\pi - 3{,}14|}{\pi} = \frac{0{,}00159\ldots}{3{,}14159\ldots} \approx 5{,}07 \times 10^{-4}.$$

Buscamos el mayor t tal que $5{,}07 \times 10^{-4} \leq 5 \times 10^{-t}$:

$$\begin{aligned} t = 3: &\quad 5 \times 10^{-3} = 0{,}005 > 5{,}07 \times 10^{-4} \quad \checkmark \\ t = 4: &\quad 5 \times 10^{-4} = 0{,}0005 < 5{,}07 \times 10^{-4} \quad \times \end{aligned}$$

Por lo tanto $\hat{p} = 3{,}14$ posee **3 dígitos exactos** de π, lo que coincide con la observación directa: los dígitos 3, 1 y 4 son correctos.

Ejemplo 4 ($\pi \approx 3{,}1416$)**.** Con $\hat{p} = 3{,}1416$:

$$R_p = \frac{|3{,}14159\ldots - 3{,}1416|}{3{,}14159\ldots} \approx 2{,}66 \times 10^{-6}.$$

Verificamos:

$$\begin{aligned} t=5: &\quad 5\times 10^{-5} > 2{,}66\times 10^{-6} \quad \checkmark \\ t=6: &\quad 5\times 10^{-6} > 2{,}66\times 10^{-6} \quad \checkmark \\ t=7: &\quad 5\times 10^{-7} < 2{,}66\times 10^{-6} \quad \times \end{aligned}$$

La aproximación tiene **6 dígitos exactos**. Nótese que $\hat{p}$ tiene 5 dígitos escritos, pero el último (6) resulta del redondeo correcto de 59 . . .; como puede verificarse en la comparación dígito a dígito:

$$\begin{aligned} \pi &= 3{,}1415\mathbf{9}2\ldots \\ \tilde{p} &= 3{,}1415\mathbf{9}\overline{2}\ldots \quad \longrightarrow \quad 3{,}141\mathbf{6}, \end{aligned}$$

el redondeo es correcto en esa posición, y el error relativo reconoce esta exactitud adicional arrojando $t = 6$.

Ejemplo 5 (Aproximación de e)**.** Sea $p = e = 2{,}71828\ldots$ y $\hat{p} = 2{,}718$.

$$R_p = \frac{|e - 2{,}718|}{e} \approx \frac{2{,}84\times 10^{-4}}{2{,}718} \approx 1{,}04\times 10^{-4}.$$

Como $1{,}04\times 10^{-4} \leq 5\times 10^{-4}$ pero $1{,}04\times 10^{-4} > 5\times 10^{-5}$, la aproximación posee **4 dígitos exactos**.

Observación 3 (Convención alternativa)**.** El texto de Mathews [1] define el número de *cifras significativas* de $\hat{p}$ como el mayor entero t tal que

$$\frac{|p-\hat{p}|}{|p|} \leq 0.5\times 10^{-t} \;=\; 5\times 10^{-(t+1)}.$$

Esta condición es *más exigente* que la de la Definición 1.3: el t de Mathews equivale al $(t+1)$ de Burden, o equivalentemente, el t de Burden equivale al $(t-1)$ de Mthews.

Consecuencia práctica. Aplicando la convención de Mathews a $\pi \approx 3{,}14$ se obtiene $t = 2$ cifras significativas, mientras que la Definición 1.3 arroja $t = 3$ dígitos exactos. Ambos resultados son correctos dentro de sus respectivas convenciones; la discrepancia es puramente de indexación.

Al consultar bibliografía, es imprescindible identificar qué convención utiliza cada autor para interpretar correctamente los resultados. En este apunte adoptamos la definición de Burden por su mayor coherencia con la noción intuitiva de "dígitos correctos".

1.2.3. Pérdida de cifras significativas

Consideremos un ejemplo:

$$x = 22.3829, \quad \hat{x} = 22.3830593, \qquad y = 22.3610, \quad \hat{y} = 22.3607291.$$

Veamos los errores absolutos y relativos:

[1] John H. Mathews, Kurtis D. Fink, *Métodos numéricos con Matlab*, Pearson Educación, Prentice-Hall, 2000

```
>> x=22.3829
x =
 22.382899999999999
>> xs = 22.3830593
xs =
 22.383059299999999
>> Ex = abs(x-xs)
Ex =
    1.592999999999734e-04
>> Rx = Ex / abs(x)
Rx =
    7.117040240539580e-06
>>
>> y = 22.3610
y =
 22.361000000000001
>> ys = 22.360791
ys =
 22.360790999999999
>> Ey = abs(y-ys)
Ey =
    2.090000000016801e-04
>> Ry = Ey/abs(y)
Ry =
    9.346630293890258e-06
```

Vemos que los dos errores relativos implican que $\hat{x}$ y $\hat{y}$ tienen 5 cifras significativas correctas. ¿Qué ocurre cuando restamos $\hat{x}$ y $\hat{y}$? Veamos

```
>> resta = x-y
resta =
  0.021899999999999
>> restas = xs-ys
restas =
  0.022268300000000
>> Eresta = abs(resta-restas)
Eresta =
    3.683000000016534e-04
>> Rresta = Eresta/abs(resta)
Rresta =
  0.016817351598250
```

Vemos que el error relativo de la resta es $0.0168\ldots$, es decir, 2 cifras significativas.

Pasamos de errores relativos de orden 10^{-6} a errores relativos de orden 10^{-2}. En términos de cifras significativas, pasamos de 5 a 2!!! Este fenómeno se conoce como *pérdida de cifras significativas*, y ocurre al restar números muy cercanos.

1.2.4. Amplificación del error de redondeo por división

Así como la resta de números cercanos produce pérdida de cifras significativas, la división por un número muy pequeño introduce un fenómeno complementario: la **amplificación del error de redondeo**.

Sea a un número exactamente representable en $\mathbb{F}$, y sea b un número cuya representación en $\mathbb{F}$ introduce un error de redondeo δb, de modo que $\tilde{b} = b + \delta b$. El error relativo del cociente es:

$$\frac{\frac{a}{b} - \frac{a}{\tilde{b}}}{\frac{a}{b}} = \frac{\tilde{b} - b}{\tilde{b}} = \frac{\tilde{b} - b}{b} \cdot \frac{1}{1 + \delta b/b} \approx \frac{\delta b}{b}, \tag{1.2}$$

cuando $|\delta b/b|$ es pequeño. Esto muestra que el error relativo del cociente es del mismo orden que el error relativo del denominador. Sin embargo, si $|b|$ es muy pequeño, un error absoluto $|\delta b|$ que parece despreciable produce un error relativo $|\delta b/b|$ muy grande, que se transfiere directamente al resultado.

En otras palabras: **dividir por un número pequeño amplifica el error absoluto del denominador**, pudiendo destruir todas las cifras significativas del resultado.

Ejemplo 6. Consideremos $a = 1$ y una sucesión de denominadores $b_k = 10^{-k}$ con errores de redondeo del orden de ε_{maq}:

```
>> a = 1;
>> for k = 1:16
     b = 10^(-k);
     bt = b + eps;
     fprintf('k=%2d, b=%.0e, error_rel=%.2e\n', ...
              k, b, abs(a/b - a/bt)/(a/b));
   end
```

Para k pequeño el error relativo es despreciable, pero a medida que $b_k \to 0$ el error relativo crece sin control, llegando a ser del orden de 1 (pérdida total de cifras significativas) cuando b es del orden de ε_{maq}.

Observación 4. Este fenómeno tiene consecuencias directas en el método de eliminación de Gauss: si en algún paso el pivote es muy pequeño, los errores de redondeo se amplifican y pueden contaminar todo el sistema. La estrategia para evitarlo es el **pivoteo parcial**: reorganizar las filas para garantizar que el pivote sea siempre el mayor elemento disponible en valor absoluto, tema que se retoma en el capítulo siguiente.

1.3. Conceptos fundamentales

Antes de estudiar métodos numéricos concretos, es necesario establecer un marco conceptual que permita evaluar su calidad. Las preguntas centrales son: ¿tiene sentido el problema que queremos resolver?, ¿qué tan sensible es su solución a perturbaciones en los datos?, y ¿el método numérico elegido se comporta bien?

Trabajaremos con problemas expresados en la forma general

$$F(x, d) = 0, \tag{1.3}$$

donde x es la incógnita a determinar y d representa los datos del problema. Esta notación unifica bajo una misma expresión una amplia variedad de problemas: sistemas de ecuaciones lineales y no lineales, ecuaciones diferenciales, problemas de interpolación, entre otros. El operador F describe la relación entre los datos y la incógnita, sin hacer referencia aún a ningún método de resolución.

1.3.1. Problema bien planteado

Un método numérico no puede ser mejor que el problema que intenta resolver. El primer paso es entonces preguntarse si el problema está bien formulado.

Definición 1.4 (Problema bien planteado)**.** El problema (1.3) se dice **bien planteado** (en el sentido de Hadamard) si satisface las tres condiciones siguientes:

1. **Existencia:** el problema tiene al menos una solución x.

2. **Unicidad:** la solución x es única para cada dato d.

3. **Dependencia continua:** la solución x varía de forma continua con los datos d; es decir, pequeñas perturbaciones en d producen pequeñas variaciones en x.

Un problema que no satisface alguna de estas condiciones se denomina **mal planteado**.

La tercera condición es la más relevante en la práctica computacional: si una pequeña variación en d (inevitable por errores de redondeo o de medición) provoca una variación enorme en x, el problema es matemáticamente legítimo pero numéricamente inútil.

Ejemplo 1 (Sistema compatible determinado)**.** El sistema lineal

$$\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$$

tiene solución única $(x, y) = (1, 3)$. Si perturbamos levemente el dato $d = (5,\ 10)$ a $\tilde{d} = (5{,}01,\ 10{,}01)$, la solución varía a $(1{,}004,\ 2{,}999)$, una variación pequeña. El problema es bien planteado.

Ejemplo 2 (Sistema casi singular)**.** Consideremos el sistema

$$\begin{pmatrix} 1 & 1 \\ 1 & 1{,}001 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

cuya solución es $(x, y) = (2, 0)$. Si perturbamos el dato a $\tilde{d} = (2,\ 2{,}001)$, la solución pasa a ser $(x, y) = (1, 1)$: una perturbación del $0{,}025\,\%$ en d produjo una variación del $100\,\%$ en x. El problema es formalmente bien planteado, pero su comportamiento ante perturbaciones en d es alarmante. Esto motiva el concepto de condicionamiento.

1.3.2. Condicionamiento

El condicionamiento mide cuantitativamente la sensibilidad de la solución de (1.3) ante perturbaciones en los datos d, independientemente del método numérico utilizado. Es una propiedad del *problema*, no del *método*.

Observación 5 (Alcance de esta sección)**.** El concepto de condicionamiento es general y aplica a cualquier problema de la forma (1.3). En este curso nos limitamos al caso particular de sistemas de ecuaciones lineales, donde el condicionamiento se cuantifica mediante el número de condición de la matriz del sistema.

Definición 1.5 (Número de condición)**.** Para una matriz A invertible, el **número de condición** se define como

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|,$$

donde $\|\cdot\|$ es una norma matricial consistente. Se tiene siempre $\kappa(A) \geq 1$.

El número de condición cuantifica el peor caso posible de amplificación del error relativo: si los datos d tienen un error relativo del orden de ε, la solución puede tener un error relativo del orden de $\kappa(A) \cdot \varepsilon$.

Observación 6 (Interpretación práctica)**.** Si $\kappa(A)$ es del orden de 10^k, se pierden aproximadamente k dígitos de precisión al resolver el sistema. Por ejemplo, si se trabaja con aritmética de 15 dígitos decimales (como en precisión doble de punto flotante) y $\kappa(A) \approx 10^6$, la solución tendrá aproximadamente $15 - 6 = 9$ dígitos confiables. Un problema con $\kappa(A)$ grande se denomina **mal condicionado**.

Ejemplo 3 (Condicionamiento del sistema casi singular)**.** Para la matriz del Ejemplo 2, puede verificarse que $\kappa(A) \approx 4000$, lo que explica la enorme amplificación del error observada: una perturbación relativa de 10^{-5} en los datos se amplificó en un factor de $\sim 10^3$.

1.3.3. Consistencia, estabilidad y convergencia

Una vez que el problema (1.3) está bien planteado, debemos evaluar la calidad del método numérico elegido para resolverlo. Todo método numérico construye una sucesión de problemas aproximados (1.4) cuyas soluciones x_n se esperan cercanas a la solución de (1.3) x. Los tres conceptos siguientes caracterizan esta calidad: la **consistencia** asegura que los problemas aproximados representen fielmente al original, la **estabilidad** asegura que el método no amplifique perturbaciones en los datos, y la **convergencia** es el resultado que se obtiene cuando ambas condiciones se satisfacen.

Consistencia

Un método numérico construye una sucesión de problemas aproximados

$$F_n(x_n, d_n) = 0, \quad n = 1, 2, \ldots \tag{1.4}$$

donde F_n, x_n y d_n son aproximaciones de F, x y d respectivamente, que dependen del paso o nivel de discretización n.

Definición 1.6 (Consistencia)**.** El método (1.4) es **consistente** con el problema (1.3) si, cuando los datos aproximados convergen a los datos exactos ($d_n \to d$), el operador aproximado converge al operador original:

$$\lim_{n\to\infty} F_n(x_n, d_n) = F(x, d) \quad \text{para todo } x_n \to x.$$

En particular, la solución exacta x satisface $\lim_{n\to\infty} F_n(x_n, d_n) = 0$.

La consistencia garantiza que el problema aproximado (1.4) es una representación legítima del problema original (1.3) cuando n crece. Sin consistencia, el método podría converger a una solución que no tiene relación con el problema original.

Ejemplo 4 (Método de Newton para $f(x) = 0$)**.** Para resolver $F(x, d) = f(x) = 0$, el método de Newton genera la sucesión

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)},$$

lo que equivale a resolver en cada paso el problema aproximado

$$F_n(x_{n+1}, d_n) = f(x_n) + f'(x_n)(x_{n+1} - x_n) = 0,$$

es decir, reemplazar f por su linealización en x_n. Como $f'(x_n)(x_{n+1} - x_n) + f(x_n) \to f(x) = 0$ cuando $x_n \to x$, el método es consistente.

Ejemplo 5 (Iteración de punto fijo)**.** Para resolver $F(x, d) = x - g(x) = 0$, el método de punto fijo define $x_{n+1} = g(x_n)$, lo que equivale a

$$F_n(x_{n+1}, d_n) = x_{n+1} - g(x_n) = 0.$$

La consistencia se verifica porque si $x_n \to x$, entonces $F_n(x, d_n) = x - g(x) = 0$, que es exactamente $F(x, d) = 0$.

Estabilidad

Definición 1.7 (Estabilidad)**.** El método (1.4) es **estable** si pequeñas perturbaciones en los datos d_n producen variaciones acotadas en la solución aproximada x_n. Formalmente, existen constantes $C > 0$ y $\delta > 0$ tales que si $\|\delta d_n\| < \delta$, entonces la solución $\tilde{x}_n$ del problema perturbado

$$F_n(\tilde{x}_n,\, d_n + \delta d_n) = 0$$

satisface

$$\|x_n - \tilde{x}_n\| \leq C\, \|\delta d_n\| \quad \text{para todo } n.$$

La estabilidad captura la robustez del método ante los errores de redondeo que se acumulan en d_n en cada paso. Un método inestable amplifica estas perturbaciones de forma descontrolada, alejando x_n de la solución exacta x aunque el método sea consistente.

Observación 7 (Consistencia sin estabilidad)**.** Un método puede ser consistente pero inestable. En ese caso, aunque $F_n(x_n, d_n) \to F(x, d) = 0$, las perturbaciones en d_n se amplifican a lo largo de las iteraciones y x_n diverge de x. La consistencia garantiza *hacia dónde* apunta el método; la estabilidad garantiza que el método *efectivamente llega*.

Convergencia

Definición 1.8 (Convergencia)**.** El método (1.4) es **convergente** si, para una elección adecuada del dato inicial d_0, la sucesión $\{x_n\}$ se aproxima a la solución exacta x de (1.3):

$$\lim_{n\to\infty} x_n = x.$$

La convergencia es el objetivo final: que el método efectivamente calcule la solución. La relación entre los tres conceptos puede resumirse en el siguiente resultado central:

Observación 8 (Relación entre los tres conceptos)**.** Para una amplia clase de métodos iterativos bien comportados, se cumple:

$$\text{consistencia} + \text{estabilidad} \implies \text{convergencia}.$$

Este resultado justifica por qué se estudian consistencia y estabilidad por separado: son las dos condiciones *verificables* cuya combinación garantiza la convergencia, que es la condición *deseable* pero más difícil de demostrar directamente.